# Fast matrix multiplication using CP decomposition

Computational laboratory project report

Alberto Defendi

**Abstract**

Bla bla bla

## Contents



## 1 Notation and tensor preliminaries

A tensor is a multidimensional array, and in this work we deal only with 3-dimenisional (or 3-way) tensors $\mathcal{X} \in \mathbb{R}^{m \times n \times p}$.

Short recap of tensor stuff

## 2 Low rank tensor decomposition

## 3 Fast matrix multiplication

Matrix multiplication is a bilinear operation. We indicate with $\langle m, n, p \rangle$ the bilinear form that represents the multiplication between a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $\mathbf{B} \in \mathbb{R}^{n \times p}$. Fixing the natural ordering on the entries of $\mathbf{A}$ and $\mathbf{B}$ and the inverse ordering on $\mathbf{C} = \mathbf{AB}$, we get a representation of $\mathcal{X} \in \mathbb{R}^{mn \times np \times mp}$ of the matrix multiplication $\langle m, n, p \rangle$ such that

$$\mathrm{vec}(\mathbf{C}^\top) = \mathcal{X} \times_1 \mathrm{vec}(\mathbf{A})^\top \times_2 \mathrm{vec}(\mathbf{B})^\top. \tag{1}$$

Note that we fix the linear order on $A$ and $B$, while we fix the reverse ordering on $C$ by applying the transpose operation.

why

Tensor definition

If we are given a CP decomposition of rank $r$ of the tensor $\mathcal{X} = [\![U, V, W]\!]$, we can rewrite Equation 1 into

$$\text{(2)} \qquad \operatorname{vec}(\mathbf{C}^\top) = \sum_{l=1}^{r} \underbrace{(\mathbf{u_l}^\top \operatorname{vec}(\mathbf{A}))}_{s_l} \cdot \underbrace{(\mathbf{v_l}^\top \operatorname{vec}(\mathbf{B}))}_{t_l} \mathbf{w_l} = \sum_{l=1}^{r} \underbrace{(s_l \cdot t_l)}_{m_l} w_l,$$

where $\mathbf{u_l}$, $\mathbf{v_l}$, $\mathbf{w_l}$ denote the columns of $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$. This reduces the number of active multiplications to the rank $r$ of $\mathcal{X}$.

When $m, n, p > 2$ we can always partition them in blocks as:

$$\text{(3)} \qquad \begin{array}{c} \\ \lfloor m/2 \rfloor \{ \\ \lceil m/2 \rceil \{ \end{array} \begin{array}{c} \overbrace{\phantom{A_{11}}}^{\lfloor n/2 \rfloor} \ \overbrace{\phantom{A_{12}}}^{\lceil n/2 \rceil} \\ \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \end{array}, \qquad \begin{array}{c} \\ \lfloor n/2 \rfloor \{ \\ \lceil n/2 \rceil \{ \end{array} \begin{array}{c} \overbrace{\phantom{B_{11}}}^{\lfloor p/2 \rfloor} \ \overbrace{\phantom{B_{12}}}^{\lceil p/2 \rceil} \\ \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \end{array},$$

where $A_{11} \in \mathbb{R}^{\lfloor \frac{m}{2} \rfloor \times \lfloor \frac{n}{2} \rfloor}$, $B_{11} \in \mathbb{R}^{\lfloor \frac{n}{2} \rfloor \times \lfloor \frac{p}{2} \rfloor}$, $A_{22} \in \mathbb{R}^{\lceil \frac{m}{2} \rceil \times \lceil \frac{n}{2} \rceil}$, and $B_{22} \in \mathbb{R}^{\lceil \frac{n}{2} \rceil \times \lceil \frac{p}{2} \rceil}$.

In particular, we can view the product $AB$ as the product between $2 \times 2$ block matrices

$$\text{(4)} \quad \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{bmatrix}.$$

The naive algorithm proceeds by combining a set of eight matrix multiplications with four additions:

$$\begin{aligned} M_1 &= A_{11} \cdot B_{11} & M_2 &= A_{12} \cdot B_{21} & M_3 &= A_{11} \cdot B_{12} \\ M_4 &= A_{12} \cdot B_{22} & M_5 &= A_{21} \cdot B_{11} & M_6 &= A_{22} \cdot B_{21} \\ M_7 &= A_{21} \cdot B_{12} & M_8 &= A_{22} \cdot B_{22} \end{aligned}$$

$$\begin{aligned} C_{11} &= M_1 + M_2 & C_{12} &= M_3 + M_4 \\ C_{21} &= M_5 + M_6 & C_{22} &= M_7 + M_8 \end{aligned}$$

For example, when $m = n = p = 2$, we get the tensor with frontal slices

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{X}_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

This yields, for example,

$$X_3 \times_1 \operatorname{vec}(A)^\top \times_2 \operatorname{vec}(B)^\top = \begin{bmatrix} a_{11} & a_{21} & a_{12} & a_{22} \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} b_{11} \\ b_{21} \\ b_{12} \\ b_{22} \end{bmatrix} = a_{21}b_{11} + a_{22}b_{21} = c_{21}.$$

Note that the formula in Equation 2 in the context of block matrix multiplication in Equation 4 yelds to

$$\mathrm{vec}(A) = \begin{bmatrix} A_{11} \\ A_{21} \\ A_{12} \\ A_{22} \end{bmatrix}, \quad \mathrm{vec}(B) = \begin{bmatrix} B_{11} \\ B_{21} \\ B_{12} \\ B_{22} \end{bmatrix}, \quad \mathrm{vec}(C^\top) = \begin{bmatrix} C_{11} \\ C_{12} \\ C_{21} \\ C_{22} \end{bmatrix}.$$

Also the factors $s_l$ and $t_l$ in Equation 2 become

$$s_l = u_l^\top \, \mathrm{vec}(A) = \sum_{i=1}^{4} U_{li} \, \mathrm{vec}(A)_i, \quad t_l = V_l^\top \, \mathrm{vec}(B) = \sum_{i=1}^{4} V_{li} \, \mathrm{vec}(B)_i$$

We follow the work [1].

Fast matrix multiplication algorithm such as Strassen face two significant problems in practical implementation, which are recursion cutoff and input matrices sizes not matching with base case. The former can solved by ... and the latter by dynamic peeling 5.

## 4

## 5 Dynamic Peeling

In Strassen algorithm we want to multiply $\langle m, n, p \rangle$ matrices by splitting in four blocks as in Equation 3. If $m$, $n$ and $p$ are even, then it is not possible to split blocks in even sizes. The easiest fix for this is padding the matrix with zeros until an even size is reached, but this adds a significant memory waste. A more efficient approach, called *dynamic peeling*, consists in stripping the extra row off and/or column and adding their contribution to the final results. In detail,

For the matrix multiplication $C = AB$, where $A$ is an $m \times n$ matrix and $B$ is an $n \times p$ matrix. The matrix $A$ is split into an $(m-1) \times (n-1)$ core matrix $A_{11}$, alongside its peeled boundary vectors $a_{12}$, $a_{21}$, and the scalar corner element $a_{22}$. Similarly, the matrix $B$ is split into an $(n-1) \times (p-1)$ core matrix $B_{11}$, with its corresponding peeled components $b_{12}$, $b_{21}$, and $b_{22}$:

$$A = \begin{array}{c} \overbrace{\hphantom{A_{11} \quad a_{12}}}^{n} \\ \left[\begin{array}{c|c} A_{11} & a_{12} \\ \hline a_{21} & a_{22} \end{array}\right] \end{array} \begin{array}{l} \\ \left.\vphantom{A_{11}}\right\} m \\ \left.\vphantom{a_{21}}\right\} 1 \end{array}, \qquad B = \begin{array}{c} \overbrace{\hphantom{B_{11} \quad b_{12}}}^{p} \\ \left[\begin{array}{c|c} B_{11} & b_{12} \\ \hline b_{21} & b_{22} \end{array}\right] \end{array} \begin{array}{l} \\ \left.\vphantom{B_{11}}\right\} n \\ \left.\vphantom{b_{21}}\right\} 1 \end{array}$$

The final block matrix product is computed directly through a combination of the core Strassen multiplication ($A_{11}B_{11}$) and low-rank vector/scalar fixup modifications:

$$\begin{array}{cl} \overbrace{\hphantom{C_{11} \quad c_{12}}}^{p} & \\ \left[\begin{array}{c|c} C_{11} & c_{12} \\ \hline c_{21} & c_{22} \end{array}\right] & \begin{array}{l} \}\, m \\ \}\, 1 \end{array} \end{array} = \left[\begin{array}{c|c} A_{11}B_{11} + a_{12}b_{21} & A_{11}b_{12} + a_{12}b_{22} \\ \hline a_{21}B_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{array}\right]$$

Here, only the sub-product $A_{11}B_{11}$ is executed recursively using Strassen's matrix multiplication, while all other terms represent the optimized boundary fixup steps.

## References

[1] Austin R. Benson and Grey Ballard. A framework for practical parallel fast matrix multiplication. *SIGPLAN Not.*, 50(8):42–53, January 2015.

[2] Austin R. Benson and Grey Ballard. A framework for practical parallel fast matrix multiplication. In *Proceedings of the 20th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP 2015, page 42–53, New York, NY, USA, 2015. Association for Computing Machinery.

[3] S. Huss-Lederman, E.M. Jacobson, J.R. Johnson, A. Tsao, and T. Turnbull. Implementation of strassen's algorithm for matrix multiplication. In *Supercomputing '96:Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, pages 32–32, 1996.